Contents

1 Chats

Exported on: 3/17/2026, 12:34:53 PM

1.1 👤 You

You said

Generate an entity relationship diagram from code below, use landscape format for a POSTER!

Derice Header frpom comments!

– Neural Network in PostgreSQL - Core Schema

– Sparse data structures for vectors and matrices

– Sparse vector metadata

CREATE TABLE sparse_vectors (

vector_id SERIAL PRIMARY KEY,

size BIGINT NOT NULL CHECK (size > 0),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

metadata JSONB DEFAULT '{}'::jsonb,

UNIQUE(vector_id, size)

);

– Sparse vector elements (only non-zero values stored)

CREATE TABLE sparse_vector_elements (

vector_id INTEGER NOT NULL REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,

index BIGINT NOT NULL CHECK (index >= 0),

value DOUBLE PRECISION NOT NULL,

PRIMARY KEY (vector_id, index),

CHECK (index < (SELECT size FROM sparse_vectors WHERE vector_id = sparse_vector_elements.vector_id))

);

– Sparse matrix metadata

CREATE TABLE sparse_matrices (

matrix_id SERIAL PRIMARY KEY,

rows BIGINT NOT NULL CHECK (rows > 0),

cols BIGINT NOT NULL CHECK (cols > 0),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

metadata JSONB DEFAULT '{}'::jsonb,

UNIQUE(matrix_id, rows, cols)

);

– Sparse matrix elements (only non-zero values stored)

CREATE TABLE sparse_matrix_elements (

matrix_id INTEGER NOT NULL REFERENCES sparse_matrices(matrix_id) ON DELETE CASCADE,

row BIGINT NOT NULL CHECK (row >= 0),

col BIGINT NOT NULL CHECK (col >= 0),

value DOUBLE PRECISION NOT NULL,

PRIMARY KEY (matrix_id, row, col),

CHECK (row < (SELECT rows FROM sparse_matrices WHERE matrix_id = sparse_matrix_elements.matrix_id)),

CHECK (col < (SELECT cols FROM sparse_matrices WHERE matrix_id = sparse_matrix_elements.matrix_id))

);

– Layer type registry (extensible system)

CREATE TABLE layer_types (

type_id SERIAL PRIMARY KEY,

name VARCHAR(100) UNIQUE NOT NULL,

description TEXT,

forward_function VARCHAR(100) NOT NULL, – PL/pgSQL function name

backward_function VARCHAR(100) NOT NULL, – PL/pgSQL function name

parameter_schema JSONB DEFAULT '{}'::jsonb, – JSON schema for layer parameters

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

– Neural network definitions

CREATE TABLE networks (

network_id SERIAL PRIMARY KEY,

name VARCHAR(200) UNIQUE NOT NULL,

description TEXT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

config JSONB DEFAULT '{}'::jsonb – global config (learning rate, etc.)

);

– Layers in a network (ordered by layer_index)

CREATE TABLE layers (

layer_id SERIAL PRIMARY KEY,

network_id INTEGER NOT NULL REFERENCES networks(network_id) ON DELETE CASCADE,

layer_index INTEGER NOT NULL CHECK (layer_index >= 0),

type_id INTEGER NOT NULL REFERENCES layer_types(type_id),

parameters JSONB DEFAULT '{}'::jsonb, – layer-specific parameters

weights_matrix_id INTEGER REFERENCES sparse_matrices(matrix_id) ON DELETE SET NULL,

biases_vector_id INTEGER REFERENCES sparse_vectors(vector_id) ON DELETE SET NULL,

activation VARCHAR(50) DEFAULT 'relu', – activation function name

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

UNIQUE(network_id, layer_index),

CHECK (layer_index >= 0)

);

– Datasets for training/evaluation

CREATE TABLE datasets (

dataset_id SERIAL PRIMARY KEY,

name VARCHAR(200) UNIQUE NOT NULL,

description TEXT,

feature_dim BIGINT NOT NULL CHECK (feature_dim > 0),

label_dim BIGINT NOT NULL CHECK (label_dim > 0),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

metadata JSONB DEFAULT '{}'::jsonb

);

– Samples (features stored as sparse vectors)

CREATE TABLE samples (

sample_id SERIAL PRIMARY KEY,

dataset_id INTEGER NOT NULL REFERENCES datasets(dataset_id) ON DELETE CASCADE,

label INTEGER NOT NULL CHECK (label >= 0),

feature_vector_id INTEGER NOT NULL REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,

split VARCHAR(20) DEFAULT 'train' CHECK (split IN ('train', 'val', 'test')),

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Gradient accumulators for training
CREATE TABLE gradient_accumulators (
accumulator_id SERIAL PRIMARY KEY,
layer_id INTEGER NOT NULL REFERENCES layers(layer_id) ON DELETE CASCADE,
weight_grad_matrix_id INTEGER REFERENCES sparse_matrices(matrix_id) ON DELETE CASCADE,
bias_grad_vector_id INTEGER REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,
batch_count INTEGER DEFAULT 0,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE(layer_id)
);
-- Forward pass cache (stores intermediate values for backprop)
CREATE TABLE forward_cache (
cache_id SERIAL PRIMARY KEY,
layer_id INTEGER NOT NULL REFERENCES layers(layer_id) ON DELETE CASCADE,
sample_id INTEGER NOT NULL REFERENCES samples(sample_id) ON DELETE CASCADE,
input_vector_id INTEGER NOT NULL REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,
preact_vector_id INTEGER NOT NULL REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,
output_vector_id INTEGER NOT NULL REFERENCES sparse_vectors(vector_id) ON DELETE CASCADE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE(layer_id, sample_id)
);
-- Training runs (track experiments)
CREATE TABLE training_runs (
run_id SERIAL PRIMARY KEY,
network_id INTEGER NOT NULL REFERENCES networks(network_id) ON DELETE CASCADE,
dataset_id INTEGER NOT NULL REFERENCES datasets(dataset_id) ON DELETE CASCADE,
start_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
end_time TIMESTAMP,
```

config JSONB DEFAULT '{}'::jsonb, – hyperparameters

status VARCHAR(50) DEFAULT 'running' CHECK (status IN ('running', 'completed', 'failed', 'stopped')),

metrics JSONB DEFAULT '{}'::jsonb – final metrics

);

– Epoch metrics (track progress)

CREATE TABLE epoch_metrics (

epoch_id SERIAL PRIMARY KEY,

run_id INTEGER NOT NULL REFERENCES training_runs(run_id) ON DELETE CASCADE,

epoch INTEGER NOT NULL CHECK (epoch >= 0),

train_loss DOUBLE PRECISION,

val_loss DOUBLE PRECISION,

train_accuracy DOUBLE PRECISION,

val_accuracy DOUBLE PRECISION,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

UNIQUE(run_id, epoch)

);

– Indexes for performance

CREATE INDEX idx_sparse_vector_elements_vector ON sparse_vector_elements(vector_id);

CREATE INDEX idx_sparse_vector_elements_index ON sparse_vector_elements(vector_id, index);

CREATE INDEX idx_sparse_matrix_elements_matrix ON sparse_matrix_elements(matrix_id);

CREATE INDEX idx_sparse_matrix_elements_row_col ON sparse_matrix_elements(matrix_id, row, col);

CREATE INDEX idx_layers_network ON layers(network_id);

CREATE INDEX idx_samples_dataset_split ON samples(dataset_id, split);

CREATE INDEX idx_samples_feature_vector ON samples(feature_vector_id);

CREATE INDEX idx_epoch_metrics_run ON epoch_metrics(run_id);– Sparse linear algebra operations for neural networks

– All operations return IDs of newly created sparse vectors/matrices

– Create a sparse vector from an array (dense representation)

CREATE OR REPLACE FUNCTION sparse_vector_from_array(

arr DOUBLE PRECISION[],

metadata JSONB DEFAULT '{}'::jsonb

) RETURNS INTEGER AS $$

```
DECLARE
v_id INTEGER;
i INTEGER;
val DOUBLE PRECISION;
BEGIN
– Create vector metadata
INSERT INTO sparse_vectors (size, metadata)
VALUES (array_length(arr, 1), metadata)
RETURNING vector_id INTO v_id;
– Insert non-zero elements
FOR i IN 1..array_length(arr, 1) LOOP
val := arr[i];
IF val != 0.0 THEN
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (v_id, i - 1, val); – 0-indexed
END IF;
END LOOP;
RETURN v_id;
END;
$$ LANGUAGE plpgsql;
– Create a sparse matrix from a 2D array (row-major)
CREATE OR REPLACE FUNCTION sparse_matrix_from_array(
mat DOUBLE PRECISION[][],
metadata JSONB DEFAULT '{}'::jsonb
) RETURNS INTEGER AS $$
DECLARE
m_id INTEGER;
r INTEGER;
c INTEGER;
val DOUBLE PRECISION;
num_rows INTEGER;
num_cols INTEGER;
BEGIN
num_rows := array_length(mat, 1);
```

```
num_cols := array_length(mat, 2);
INSERT INTO sparse_matrices (rows, cols, metadata)
VALUES (num_rows, num_cols, metadata)
RETURNING matrix_id INTO m_id;
FOR r IN 1..num_rows LOOP
FOR c IN 1..num_cols LOOP
val := mat[r][c];
IF val != 0.0 THEN
INSERT INTO sparse_matrix_elements (matrix_id, row, col, value)
VALUES (m_id, r - 1, c - 1, val); – 0-indexed
END IF;
END LOOP;
END LOOP;
RETURN m_id;
END;
$$ LANGUAGE plpgsql;
– Sparse matrix-vector multiplication: result = M * v
CREATE OR REPLACE FUNCTION sparse_matvec_mult(
matrix_id INTEGER,
vector_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
m_rows BIGINT;
m_cols BIGINT;
v_size BIGINT;
result_id INTEGER;
BEGIN
– Get dimensions
SELECT rows, cols INTO m_rows, m_cols
FROM sparse_matrices WHERE matrix_id = $1;
SELECT size INTO v_size
FROM sparse_vectors WHERE vector_id = $2;
– Validate dimensions
IF m_cols != v_size THEN
```

```
RAISE EXCEPTION 'Dimension mismatch: matrix cols % != vector size %', m_cols, v_size;
END IF;
-- Create result vector
INSERT INTO sparse_vectors (size, metadata)
VALUES (m_rows, jsonb_build_object(
'operation', 'matvec_mult',
'matrix_id', matrix_id,
'vector_id', vector_id,
'created_at', CURRENT_TIMESTAMP
))
RETURNING vector_id INTO result_id;
-- Compute product: sum over columns where matrix.col = vector.index
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, m.row AS index, SUM(m.value * v.value) AS value
FROM sparse_matrix_elements m
JOIN sparse_vector_elements v ON m.col = v.index
WHERE m.matrix_id = $1 AND v.vector_id = $2
GROUP BY m.row
HAVING ABS(SUM(m.value * v.value)) > 1e-15; -- ignore near-zero results
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
-- Sparse vector addition: result = v1 + v2
CREATE OR REPLACE FUNCTION sparse_vector_add(
v1_id INTEGER,
v2_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
v1_size BIGINT;
v2_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO v1_size FROM sparse_vectors WHERE vector_id = v1_id;
SELECT size INTO v2_size FROM sparse_vectors WHERE vector_id = v2_id;
```

```
IF v1_size != v2_size THEN
RAISE EXCEPTION 'Vector size mismatch: % != %', v1_size, v2_size;
END IF;
INSERT INTO sparse_vectors (size, metadata)
VALUES (v1_size, jsonb_build_object(
'operation', 'vector_add',
'v1_id', v1_id,
'v2_id', v2_id
))
RETURNING vector_id INTO result_id;
-- Full outer join using UNION approach
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, idx, COALESCE(v1.value, 0) + COALESCE(v2.value, 0) AS value
FROM (
SELECT index AS idx, value FROM sparse_vector_elements WHERE vector_id = v1_id
) v1
FULL OUTER JOIN (
SELECT index AS idx, value FROM sparse_vector_elements WHERE vector_id = v2_id
) v2 USING (idx)
WHERE ABS(COALESCE(v1.value, 0) + COALESCE(v2.value, 0)) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
-- Scale sparse vector: result = scalar * v
CREATE OR REPLACE FUNCTION sparse_vector_scale(
vector_id INTEGER,
scalar DOUBLE PRECISION
) RETURNS INTEGER AS $$
DECLARE
v_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO v_size FROM sparse_vectors WHERE vector_id = $1;
INSERT INTO sparse_vectors (size, metadata)
```

```
VALUES (v_size, jsonb_build_object(
'operation', 'vector_scale',
'original_id', vector_id,
'scalar', scalar
))
RETURNING vector_id INTO result_id;
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, value * scalar
FROM sparse_vector_elements
WHERE vector_id = $1
AND ABS(value * scalar) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Apply element-wise function to sparse vector
CREATE OR REPLACE FUNCTION sparse_vector_map(
vector_id INTEGER,
func_name VARCHAR(50)  – 'relu', 'sigmoid', 'tanh', 'softmax'
) RETURNS INTEGER AS $$
DECLARE
v_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO v_size FROM sparse_vectors WHERE vector_id = $1;
INSERT INTO sparse_vectors (size, metadata)
VALUES (v_size, jsonb_build_object(
'operation', 'vector_map',
'original_id', vector_id,
'function', func_name
))
RETURNING vector_id INTO result_id;
IF func_name = 'relu' THEN
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, GREATEST(value, 0.0)
```

```
FROM sparse_vector_elements
WHERE vector_id = $1
AND GREATEST(value, 0.0) > 1e-15;
ELSIF func_name = 'sigmoid' THEN
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, 1.0 / (1.0 + exp(-value))
FROM sparse_vector_elements
WHERE vector_id = $1;
ELSIF func_name = 'tanh' THEN
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, tanh(value)
FROM sparse_vector_elements
WHERE vector_id = $1;
ELSIF func_name = 'softmax' THEN
– Softmax requires all elements (including zeros), need full vector
– This is more complex, implement later
RAISE EXCEPTION 'Softmax not yet implemented for sparse vectors';
ELSE
RAISE EXCEPTION 'Unknown function: %', func_name;
END IF;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Compute dot product of two sparse vectors
CREATE OR REPLACE FUNCTION sparse_vector_dot(
v1_id INTEGER,
v2_id INTEGER
) RETURNS DOUBLE PRECISION AS $$
DECLARE
dot DOUBLE PRECISION;
BEGIN
SELECT COALESCE(SUM(v1.value * v2.value), 0.0) INTO dot
FROM sparse_vector_elements v1
JOIN sparse_vector_elements v2 ON v1.index = v2.index
```

```
WHERE v1.vector_id = $1 AND v2.vector_id = $2;
RETURN dot;
END;
$$ LANGUAGE plpgsql;
– Create a sparse vector of zeros (empty elements)
CREATE OR REPLACE FUNCTION sparse_vector_zeros(
size BIGINT,
metadata JSONB DEFAULT '{}'::jsonb
) RETURNS INTEGER AS $$
DECLARE
v_id INTEGER;
BEGIN
INSERT INTO sparse_vectors (size, metadata)
VALUES (size, metadata)
RETURNING vector_id INTO v_id;
RETURN v_id;
END;
$$ LANGUAGE plpgsql;
– Create a sparse matrix of zeros (empty elements)
CREATE OR REPLACE FUNCTION sparse_matrix_zeros(
rows BIGINT,
cols BIGINT,
metadata JSONB DEFAULT '{}'::jsonb
) RETURNS INTEGER AS $$
DECLARE
m_id INTEGER;
BEGIN
INSERT INTO sparse_matrices (rows, cols, metadata)
VALUES (rows, cols, metadata)
RETURNING matrix_id INTO m_id;
RETURN m_id;
END;
$$ LANGUAGE plpgsql;
– Sparse matrix addition: result = M1 + M2
```

```
CREATE OR REPLACE FUNCTION sparse_matrix_add(
m1_id INTEGER,
m2_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
m1_rows BIGINT;
m1_cols BIGINT;
m2_rows BIGINT;
m2_cols BIGINT;
result_id INTEGER;
BEGIN
SELECT rows, cols INTO m1_rows, m1_cols FROM sparse_matrices WHERE matrix_id = m1_id;
SELECT rows, cols INTO m2_rows, m2_cols FROM sparse_matrices WHERE matrix_id = m2_id;
IF m1_rows != m2_rows OR m1_cols != m2_cols THEN
RAISE EXCEPTION 'Matrix dimension mismatch: (% rows, % cols) != (% rows, % cols)',
m1_rows, m1_cols, m2_rows, m2_cols;
END IF;
INSERT INTO sparse_matrices (rows, cols, metadata)
VALUES (m1_rows, m1_cols, jsonb_build_object(
'operation', 'matrix_add',
'm1_id', m1_id,
'm2_id', m2_id
))
RETURNING matrix_id INTO result_id;
– Full outer join of matrix elements
INSERT INTO sparse_matrix_elements (matrix_id, row, col, value)
SELECT result_id,
COALESCE(m1.row, m2.row) AS row,
COALESCE(m1.col, m2.col) AS col,
COALESCE(m1.value, 0) + COALESCE(m2.value, 0) AS value
FROM (
SELECT row, col, value FROM sparse_matrix_elements WHERE matrix_id = m1_id
```

```
) m1
FULL OUTER JOIN (
SELECT row, col, value FROM sparse_matrix_elements WHERE matrix_id = m2_id
) m2 USING (row, col)
WHERE ABS(COALESCE(m1.value, 0) + COALESCE(m2.value, 0)) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Scale sparse matrix: result = scalar * M
CREATE OR REPLACE FUNCTION sparse_matrix_scale(
matrix_id INTEGER,
scalar DOUBLE PRECISION
) RETURNS INTEGER AS $$
DECLARE
m_rows BIGINT;
m_cols BIGINT;
result_id INTEGER;
BEGIN
SELECT rows, cols INTO m_rows, m_cols FROM sparse_matrices WHERE matrix_id = $1;
INSERT INTO sparse_matrices (rows, cols, metadata)
VALUES (m_rows, m_cols, jsonb_build_object(
'operation', 'matrix_scale',
'original_id', matrix_id,
'scalar', scalar
))
RETURNING matrix_id INTO result_id;
INSERT INTO sparse_matrix_elements (matrix_id, row, col, value)
SELECT result_id, row, col, value * scalar
FROM sparse_matrix_elements
WHERE matrix_id = $1
AND ABS(value * scalar) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
```

```
– Clean up old sparse vectors/matrices (optional)
CREATE OR REPLACE FUNCTION cleanup_sparse_data(
older_than INTERVAL DEFAULT '1 day'
) RETURNS INTEGER AS $$
DECLARE
deleted_count INTEGER := 0;
BEGIN
– Delete vectors not referenced elsewhere
WITH deletable_vectors AS (
SELECT vector_id FROM sparse_vectors
WHERE created_at < CURRENT_TIMESTAMP - older_than
AND vector_id NOT IN (
SELECT feature_vector_id FROM samples
UNION
SELECT biases_vector_id FROM layers WHERE biases_vector_id IS NOT NULL
UNION
SELECT bias_grad_vector_id FROM gradient_accumulators WHERE bias_grad_vector_id IS
NOT NULL
)
)
DELETE FROM sparse_vectors
WHERE vector_id IN (SELECT vector_id FROM deletable_vectors);
GET DIAGNOSTICS deleted_count = ROW_COUNT;
– Similar for matrices (simplified)
RETURN deleted_count;
END;
$$ LANGUAGE plpgsql;– Layer-type functions for neural networks
– Dense (fully connected) layer implementation
– Forward pass for dense layer
CREATE OR REPLACE FUNCTION forward_dense(
layer_id INTEGER,
input_vector_id INTEGER,
sample_id INTEGER DEFAULT NULL
) RETURNS INTEGER AS $$
```

```
DECLARE
weight_m_id INTEGER;
bias_v_id INTEGER;
activation_name VARCHAR(50);
preact_id INTEGER;
output_id INTEGER;
cache_rec RECORD;
BEGIN
– Get layer parameters
SELECT weights_matrix_id, biases_vector_id, activation
INTO weight_m_id, bias_v_id, activation_name
FROM layers WHERE layer_id = $1;
IF weight_m_id IS NULL OR bias_v_id IS NULL THEN
RAISE EXCEPTION 'Layer % not properly initialized (missing weights/biases)', layer_id;
END IF;
– Compute pre-activation: z = Wx + b
preact_id := sparse_matvec_mult(weight_m_id, input_vector_id);
preact_id := sparse_vector_add(preact_id, bias_v_id); – modifies preact_id
– Apply activation function
output_id := sparse_vector_map(preact_id, activation_name);
– Store cache for backward pass if sample_id provided
IF sample_id IS NOT NULL THEN
INSERT INTO forward_cache (layer_id, sample_id, input_vector_id, preact_vector_id, out-
put_vector_id)
VALUES (layer_id, sample_id, input_vector_id, preact_id, output_id)
ON CONFLICT (layer_id, sample_id) DO UPDATE SET
input_vector_id = EXCLUDED.input_vector_id,
preact_vector_id = EXCLUDED.preact_vector_id,
output_vector_id = EXCLUDED.output_vector_id,
created_at = CURRENT_TIMESTAMP;
END IF;
RETURN output_id;
END;
$$ LANGUAGE plpgsql;
```

```
– Backward pass for dense layer
– Returns (weight_grad_matrix_id, bias_grad_vector_id, input_grad_vector_id)
CREATE OR REPLACE FUNCTION backward_dense(
layer_id INTEGER,
sample_id INTEGER,
upstream_grad_vector_id INTEGER  – gradient of loss w.r.t. output
) RETURNS TABLE (
weight_grad_matrix_id INTEGER,
bias_grad_vector_id INTEGER,
input_grad_vector_id INTEGER
) AS $$
DECLARE
weight_m_id INTEGER;
bias_v_id INTEGER;
activation_name VARCHAR(50);
input_v_id INTEGER;
preact_v_id INTEGER;
output_v_id INTEGER;
activation_grad_id INTEGER;
delta_id INTEGER;  – gradient w.r.t. pre-activation
input_grad_id INTEGER;
bias_grad_id INTEGER;
weight_grad_id INTEGER;
rows BIGINT;
cols BIGINT;
BEGIN
– Get layer parameters and cached values
SELECT l.weights_matrix_id, l.biases_vector_id, l.activation,
fc.input_vector_id, fc.preact_vector_id, fc.output_vector_id
INTO weight_m_id, bias_v_id, activation_name,
input_v_id, preact_v_id, output_v_id
FROM layers l
LEFT JOIN forward_cache fc ON l.layer_id = fc.layer_id AND fc.sample_id = $2
WHERE l.layer_id = $1;
```

```
IF weight_m_id IS NULL OR bias_v_id IS NULL THEN
RAISE EXCEPTION 'Layer % not properly initialized', layer_id;
END IF;
IF input_v_id IS NULL THEN
RAISE EXCEPTION 'No forward cache found for layer %, sample %', layer_id, sample_id;
END IF;
– Compute gradient of activation w.r.t. pre-activation
activation_grad_id := activation_gradient(preact_v_id, activation_name);
– Element-wise multiply: delta = upstream_grad ⊙ activation_grad
delta_id := sparse_vector_elementwise_mul(upstream_grad_vector_id, activation_grad_id);
– Gradient w.r.t. biases: sum over batch (just delta for single sample)
bias_grad_id := delta_id;  – for single sample, bias gradient = delta
– Gradient w.r.t. weights: delta * input^T
weight_grad_id := sparse_outer_product(delta_id, input_v_id);
– Gradient w.r.t. input: W^T * delta
input_grad_id := sparse_matvec_mult_transpose(weight_m_id, delta_id);
RETURN QUERY SELECT weight_grad_id, bias_grad_id, input_grad_id;
END;
$$ LANGUAGE plpgsql;
– Helper: element-wise multiplication of two sparse vectors
CREATE OR REPLACE FUNCTION sparse_vector_elementwise_mul(
v1_id INTEGER,
v2_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
v1_size BIGINT;
v2_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO v1_size FROM sparse_vectors WHERE vector_id = v1_id;
SELECT size INTO v2_size FROM sparse_vectors WHERE vector_id = v2_id;
IF v1_size != v2_size THEN
RAISE EXCEPTION 'Vector size mismatch: % != %', v1_size, v2_size;
END IF;
```

```
INSERT INTO sparse_vectors (size, metadata)
VALUES (v1_size, jsonb_build_object(
'operation', 'elementwise_mul',
'v1_id', v1_id,
'v2_id', v2_id
))
RETURNING vector_id INTO result_id;
– Multiply matching indices (inner join)
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, v1.index, v1.value * v2.value
FROM sparse_vector_elements v1
JOIN sparse_vector_elements v2 ON v1.index = v2.index
WHERE v1.vector_id = v1_id AND v2.vector_id = v2_id
AND ABS(v1.value * v2.value) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Helper: outer product of two sparse vectors (matrix)
CREATE OR REPLACE FUNCTION sparse_outer_product(
v1_id INTEGER,
v2_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
v1_size BIGINT;
v2_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO v1_size FROM sparse_vectors WHERE vector_id = v1_id;
SELECT size INTO v2_size FROM sparse_vectors WHERE vector_id = v2_id;
INSERT INTO sparse_matrices (rows, cols, metadata)
VALUES (v1_size, v2_size, jsonb_build_object(
'operation', 'outer_product',
'v1_id', v1_id,
'v2_id', v2_id
```

))
RETURNING matrix_id INTO result_id;
– Compute outer product: for each non-zero in v1 and v2
INSERT INTO sparse_matrix_elements (matrix_id, row, col, value)
SELECT result_id, v1.index, v2.index, v1.value * v2.value
FROM sparse_vector_elements v1
CROSS JOIN sparse_vector_elements v2
WHERE v1.vector_id = v1_id AND v2.vector_id = v2_id
AND ABS(v1.value * v2.value) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Helper: matrix-vector multiplication with transpose (W^T * v)
CREATE OR REPLACE FUNCTION sparse_matvec_mult_transpose(
matrix_id INTEGER,
vector_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
m_rows BIGINT;
m_cols BIGINT;
v_size BIGINT;
result_id INTEGER;
BEGIN
SELECT rows, cols INTO m_rows, m_cols
FROM sparse_matrices WHERE matrix_id = $1;
SELECT size INTO v_size
FROM sparse_vectors WHERE vector_id = $2;
IF m_rows != v_size THEN
RAISE EXCEPTION ‘Dimension mismatch: matrix rows % != vector size %’, m_rows, v_size;
END IF;
INSERT INTO sparse_vectors (size, metadata)
VALUES (m_cols, jsonb_build_object(
‘operation’, ‘matvec_mult_transpose’,
‘matrix_id’, matrix_id,

```
'vector_id', vector_id
))
RETURNING vector_id INTO result_id;
-- Compute W^T * v: sum over rows where matrix.row = vector.index
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, m.col AS index, SUM(m.value * v.value) AS value
FROM sparse_matrix_elements m
JOIN sparse_vector_elements v ON m.row = v.index
WHERE m.matrix_id = $1 AND v.vector_id = $2
GROUP BY m.col
HAVING ABS(SUM(m.value * v.value)) > 1e-15;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
-- Helper: activation gradient functions
CREATE OR REPLACE FUNCTION activation_gradient(
preact_vector_id INTEGER,
activation_name VARCHAR(50)
) RETURNS INTEGER AS $$
DECLARE
preact_size BIGINT;
result_id INTEGER;
BEGIN
SELECT size INTO preact_size FROM sparse_vectors WHERE vector_id = preact_vector_id;
INSERT INTO sparse_vectors (size, metadata)
VALUES (preact_size, jsonb_build_object(
'operation', 'activation_gradient',
'activation', activation_name,
'preact_id', preact_vector_id
))
RETURNING vector_id INTO result_id;
IF activation_name = 'relu' THEN
-- ReLU gradient: 1 if preact > 0, else 0
INSERT INTO sparse_vector_elements (vector_id, index, value)
```

```
SELECT result_id, index, 1.0
FROM sparse_vector_elements
WHERE vector_id = preact_vector_id
AND value > 0.0;
ELSIF activation_name = 'sigmoid' THEN
– sigmoid gradient: σ(z) * (1 - σ(z))
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index,
(1.0 / (1.0 + exp(-value))) * (1.0 - (1.0 / (1.0 + exp(-value))))
FROM sparse_vector_elements
WHERE vector_id = preact_vector_id;
ELSIF activation_name = 'tanh' THEN
– tanh gradient: 1 - tanh(z)^2
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, 1.0 - tanh(value)^2
FROM sparse_vector_elements
WHERE vector_id = preact_vector_id;
ELSIF activation_name = 'linear' OR activation_name = 'identity' THEN
– Linear activation gradient: 1 everywhere (dense)
– For sparse, we need to create entries for all indices where preact exists
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, index, 1.0
FROM sparse_vector_elements
WHERE vector_id = preact_vector_id;
ELSE
RAISE EXCEPTION 'Unknown activation function: %', activation_name;
END IF;
RETURN result_id;
END;
$$ LANGUAGE plpgsql;
– Initialize dense layer weights with He initialization (Gaussian)
CREATE OR REPLACE FUNCTION init_dense_layer_weights(
layer_id INTEGER,
input_dim INTEGER,
```

```
output_dim INTEGER,
initialization VARCHAR(50) DEFAULT 'he'
) RETURNS VOID AS $$
DECLARE
weight_m_id INTEGER;
bias_v_id INTEGER;
scale DOUBLE PRECISION;
i INTEGER;
j INTEGER;
val DOUBLE PRECISION;
weight_elements DOUBLE PRECISION[][];
bias_elements DOUBLE PRECISION[];
BEGIN
– He initialization: N(0, sqrt(2/input_dim))
IF initialization = 'he' THEN
scale := sqrt(2.0 / input_dim);
– Xavier initialization: N(0, sqrt(2/(input_dim + output_dim)))
ELSIF initialization = 'xavier' THEN
scale := sqrt(2.0 / (input_dim + output_dim));
ELSE
scale := 0.01;
END IF;
– Create weight matrix (dense for now, will be sparse if many zeros)
– For simplicity, generate dense matrix with random values
– In practice, we might want sparse initialization
weight_elements := array_fill(0.0, ARRAY[output_dim, input_dim]);
FOR i IN 1..output_dim LOOP
FOR j IN 1..input_dim LOOP
– Box-Muller transform for Gaussian random
val := scale * sqrt(-2.0 * ln(random() + 1e-15)) * cos(2.0 * pi() * random());
weight_elements[i][j] := val;
END LOOP;
END LOOP;
weight_m_id := sparse_matrix_from_array(weight_elements);
```

```
– Create bias vector (initialize to zeros)
bias_elements := array_fill(0.0, ARRAY[output_dim]);
bias_v_id := sparse_vector_from_array(bias_elements);
– Update layer with weights and biases
UPDATE layers
SET weights_matrix_id = weight_m_id,
biases_vector_id = bias_v_id
WHERE layer_id = layer_id;
END;
$$ LANGUAGE plpgsql;
– Create a new dense layer and add it to a network
CREATE OR REPLACE FUNCTION create_dense_layer(
network_id INTEGER,
layer_index INTEGER,
output_dim INTEGER,
activation VARCHAR(50) DEFAULT 'relu',
initialization VARCHAR(50) DEFAULT 'he'
) RETURNS INTEGER AS $$
DECLARE
dense_type_id INTEGER;
prev_layer_output_dim INTEGER;
new_layer_id INTEGER;
BEGIN
– Get dense layer type ID
SELECT type_id INTO dense_type_id
FROM layer_types WHERE name = 'dense';
IF dense_type_id IS NULL THEN
RAISE EXCEPTION 'Dense layer type not registered. Run init_layer_types() first.';
END IF;
– Determine input dimension from previous layer
IF layer_index = 0 THEN
– Input layer: need to specify input_dim when creating network
RAISE EXCEPTION 'Input dimension must be specified for first layer';
END IF;
```

```
SELECT l2.parameters-»‘output_dim’ INTO prev_layer_output_dim
FROM layers l1
JOIN layers l2 ON l1.network_id = l2.network_id
WHERE l1.network_id = $1 AND l2.layer_index = $2 - 1;
IF prev_layer_output_dim IS NULL THEN
RAISE EXCEPTION ‘Previous layer not found or missing output_dim parameter’;
END IF;
– Create layer record
INSERT INTO layers (network_id, layer_index, type_id, activation, parameters)
VALUES (network_id, layer_index, dense_type_id, activation,
jsonb_build_object(‘output_dim’, output_dim, ‘input_dim’, prev_layer_output_dim::INTE-
GER))
RETURNING layer_id INTO new_layer_id;
– Initialize weights
PERFORM init_dense_layer_weights(new_layer_id, prev_layer_output_dim::INTEGER, out-
put_dim, initialization);
RETURN new_layer_id;
END;
$$ LANGUAGE plpgsql;– Loss functions for neural network training
– Create one-hot vector from label index
CREATE OR REPLACE FUNCTION one_hot_vector(
label INTEGER,
num_classes INTEGER,
metadata JSONB DEFAULT ‘{}’::jsonb
) RETURNS INTEGER AS $$
DECLARE
v_id INTEGER;
BEGIN
IF label < 0 OR label >= num_classes THEN
RAISE EXCEPTION ‘Label % out of range [0, %]’, label, num_classes - 1;
END IF;
INSERT INTO sparse_vectors (size, metadata)
VALUES (num_classes, jsonb_build_object(‘one_hot_label’, label) || metadata)
RETURNING vector_id INTO v_id;
```

```
– Only one non-zero element at label index
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (v_id, label, 1.0);
RETURN v_id;
END;
$$ LANGUAGE plpgsql;
– Softmax function (returns probability vector)
CREATE OR REPLACE FUNCTION softmax(
logits_vector_id INTEGER
) RETURNS INTEGER AS $$
DECLARE
logits_size BIGINT;
max_val DOUBLE PRECISION;
exp_sum DOUBLE PRECISION;
result_id INTEGER;
exp_vals DOUBLE PRECISION[];
i INTEGER;
idx BIGINT;
val DOUBLE PRECISION;
exp_val DOUBLE PRECISION;
BEGIN
SELECT size INTO logits_size FROM sparse_vectors WHERE vector_id = logits_vector_id;
– Find maximum logit for numerical stability
SELECT COALESCE(MAX(value), 0.0) INTO max_val
FROM sparse_vector_elements WHERE vector_id = logits_vector_id;
– Compute exponentials and sum
exp_sum := 0.0;
FOR idx, val IN SELECT index, value FROM sparse_vector_elements WHERE vector_id =
logits_vector_id LOOP
exp_val := exp(val - max_val);
exp_sum := exp_sum + exp_val;
– Store in array for later insertion
exp_vals := array_append(exp_vals, exp_val);
END LOOP;
```

– Add contributions from zero elements (implicit zeros)

– For sparse representation, zero logits contribute exp(-max_val) to sum

exp_sum := exp_sum + (logits_size - (SELECT COUNT*() FROM sparse_vector_elements WHERE vector_id = logits_vector_id))* exp(-max_val);

– Create result vector

INSERT INTO sparse_vectors (size, metadata)

VALUES (logits_size, jsonb_build_object('operation', 'softmax', 'logits_id', logits_vector_id))

RETURNING vector_id INTO result_id;

– Insert non-zero probabilities (all indices have non-zero probability due to softmax)

– For efficiency, we only store probabilities above a threshold

FOR idx, val, exp_val IN

SELECT index, value, exp(value - max_val) / exp_sum

FROM sparse_vector_elements WHERE vector_id = logits_vector_id

LOOP

IF exp_val > 1e-15 THEN

INSERT INTO sparse_vector_elements (vector_id, index, value)

VALUES (result_id, idx, exp_val);

END IF;

END LOOP;

– For zero logits (not stored), probability = exp(-max_val) / exp_sum

– We could store them, but they're likely small. Skip for sparse representation.

RETURN result_id;

END;

$$ LANGUAGE plpgsql;

– Cross-entropy loss with softmax (returns loss value and gradient vector)

CREATE OR REPLACE FUNCTION softmax_cross_entropy_loss(

logits_vector_id INTEGER,

label INTEGER,

num_classes INTEGER

) RETURNS TABLE (

loss DOUBLE PRECISION,

gradient_vector_id INTEGER

) AS $$

DECLARE

```
probs_id INTEGER;
one_hot_id INTEGER;
grad_id INTEGER;
loss_val DOUBLE PRECISION;
logits_size BIGINT;
max_val DOUBLE PRECISION;
exp_sum DOUBLE PRECISION;
prob DOUBLE PRECISION;
idx BIGINT;
val DOUBLE PRECISION;
BEGIN
SELECT size INTO logits_size FROM sparse_vectors WHERE vector_id = logits_vector_id;
IF num_classes != logits_size THEN
RAISE EXCEPTION 'Num classes % != logits size %', num_classes, logits_size;
END IF;
– Numerical stable softmax and cross-entropy computation
– Find max logit
SELECT COALESCE(MAX(value), 0.0) INTO max_val
FROM sparse_vector_elements WHERE vector_id = logits_vector_id;
– Compute sum of exponentials
SELECT COALESCE(SUM(exp(value - max_val)), 0.0) INTO exp_sum
FROM sparse_vector_elements WHERE vector_id = logits_vector_id;
– Add contribution from zero logits
exp_sum := exp_sum + (logits_size - (SELECT COUNT() FROM sparse_vector_elements
WHERE vector_id = logits_vector_id))  exp(-max_val);
– Compute loss: -log(exp(logits[label] - max) / exp_sum)
– First get logit at label position
SELECT COALESCE(value, 0.0) INTO val
FROM sparse_vector_elements
WHERE vector_id = logits_vector_id AND index = label;
– Logit for label (could be zero if not stored)
loss_val := -((val - max_val) - ln(exp_sum));
– Compute gradient: probs - one_hot
– Create gradient vector
```

```
INSERT INTO sparse_vectors (size, metadata)
VALUES (logits_size, jsonb_build_object(
'operation', 'cross_entropy_gradient',
'logits_id', logits_vector_id,
'label', label
))
RETURNING vector_id INTO grad_id;
-- Gradient for non-zero logits: probs_i - (1 if i == label else 0)
FOR idx, val IN SELECT index, value FROM sparse_vector_elements WHERE vector_id = logits_vector_id LOOP
prob := exp(val - max_val) / exp_sum;
IF idx = label THEN
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (grad_id, idx, prob - 1.0);
ELSE
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (grad_id, idx, prob);
END IF;
END LOOP;
-- Gradient for zero logits (not stored): prob_i - (1 if i == label else 0)
-- Zero logits have value = 0 (implicit), prob = exp(-max_val) / exp_sum
IF label >= 0 AND label < logits_size THEN
-- Check if label is not already processed (could be zero logit)
IF NOT EXISTS (SELECT 1 FROM sparse_vector_elements WHERE vector_id = logits_vector_id AND index = label) THEN
prob := exp(-max_val) / exp_sum;
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (grad_id, label, prob - 1.0);
END IF;
END IF;
RETURN QUERY SELECT loss_val, grad_id;
END;
$$ LANGUAGE plpgsql;
-- Mean squared error loss
```

```
CREATE OR REPLACE FUNCTION mse_loss(
predictions_vector_id INTEGER,
targets_vector_id INTEGER
) RETURNS TABLE (
loss DOUBLE PRECISION,
gradient_vector_id INTEGER
) AS $$
DECLARE
pred_size BIGINT;
target_size BIGINT;
diff_id INTEGER;
loss_val DOUBLE PRECISION;
grad_id INTEGER;
BEGIN
SELECT size INTO pred_size FROM sparse_vectors WHERE vector_id = predictions_vector_id;
SELECT size INTO target_size FROM sparse_vectors WHERE vector_id = targets_vector_id;
IF pred_size != target_size THEN
RAISE EXCEPTION 'Vector size mismatch: % != %', pred_size, target_size;
END IF;
– Compute difference: predictions - targets
diff_id := sparse_vector_add(
predictions_vector_id,
sparse_vector_scale(targets_vector_id, -1.0)
);
– Loss = 0.5 * mean(diff^2)
loss_val := 0.5 * sparse_vector_dot(diff_id, diff_id) / pred_size;
– Gradient = (predictions - targets) / N
grad_id := sparse_vector_scale(diff_id, 1.0 / pred_size);
RETURN QUERY SELECT loss_val, grad_id;
END;
$$ LANGUAGE plpgsql;
– Compute accuracy for classification
CREATE OR REPLACE FUNCTION compute_accuracy(
```

```
predictions_logits_vector_id INTEGER,
label INTEGER
) RETURNS DOUBLE PRECISION AS $$
DECLARE
logits_size BIGINT;
max_val DOUBLE PRECISION;
max_idx BIGINT;
BEGIN
– Find predicted class (argmax of logits)
SELECT MAX(value) INTO max_val
FROM sparse_vector_elements
WHERE vector_id = predictions_logits_vector_id;
IF max_val IS NULL THEN
– All logits are zero (implicit)
RETURN CASE WHEN label = 0 THEN 1.0 ELSE 0.0 END;
END IF;
SELECT index INTO max_idx
FROM sparse_vector_elements
WHERE vector_id = predictions_logits_vector_id AND value = max_val
LIMIT 1;
RETURN CASE WHEN max_idx = label THEN 1.0 ELSE 0.0 END;
END;
$$ LANGUAGE plpgsql;
– Batch loss and accuracy computation
CREATE OR REPLACE FUNCTION compute_batch_metrics(
sample_ids INTEGER[],
network_id INTEGER
) RETURNS TABLE (
avg_loss DOUBLE PRECISION,
avg_accuracy DOUBLE PRECISION
) AS $$
DECLARE
total_loss DOUBLE PRECISION := 0.0;
total_accuracy DOUBLE PRECISION := 0.0;
```

```
sample_count INTEGER := 0;
s_id INTEGER;
label_val INTEGER;
feature_v_id INTEGER;
output_v_id INTEGER;
loss_val DOUBLE PRECISION;
acc_val DOUBLE PRECISION;
grad_id INTEGER;
num_classes INTEGER;
BEGIN
– Get number of classes from network output layer
SELECT l.parameters-»‘output_dim’ INTO num_classes
FROM layers l
WHERE l.network_id = $2
ORDER BY l.layer_index DESC
LIMIT 1;
IF num_classes IS NULL THEN
RAISE EXCEPTION ‘Could not determine number of classes from network’;
END IF;
FOREACH s_id IN ARRAY sample_ids LOOP
– Get sample data
SELECT s.label, s.feature_vector_id INTO label_val, feature_v_id
FROM samples s WHERE s.sample_id = s_id;
– Forward pass through network (simplified - need to implement)
– For now, placeholder
output_v_id := feature_v_id;
– Compute loss and accuracy
SELECT loss, gradient_vector_id INTO loss_val, grad_id
FROM softmax_cross_entropy_loss(output_v_id, label_val, num_classes::INTEGER);
acc_val := compute_accuracy(output_v_id, label_val);
total_loss := total_loss + loss_val;
total_accuracy := total_accuracy + acc_val;
sample_count := sample_count + 1;
END LOOP;
```

```
IF sample_count > 0 THEN
avg_loss := total_loss / sample_count;
avg_accuracy := total_accuracy / sample_count;
ELSE
avg_loss := NULL;
avg_accuracy := NULL;
END IF;
RETURN QUERY SELECT avg_loss, avg_accuracy;
END;
$$ LANGUAGE plpgsql;– Training functions for gradient descent optimization
– Accumulate gradients for a layer (add to existing accumulator)
CREATE OR REPLACE FUNCTION accumulate_gradients(
layer_id INTEGER,
weight_grad_matrix_id INTEGER,
bias_grad_vector_id INTEGER,
batch_size INTEGER DEFAULT 1
) RETURNS VOID AS $$
DECLARE
acc_rec RECORD;
existing_weight_grad_id INTEGER;
existing_bias_grad_id INTEGER;
new_weight_grad_id INTEGER;
new_bias_grad_id INTEGER;
BEGIN
– Get existing accumulator
SELECT weight_grad_matrix_id, bias_grad_vector_id, batch_count
INTO existing_weight_grad_id, existing_bias_grad_id, acc_rec
FROM gradient_accumulators
WHERE gradient_accumulators.layer_id = accumulate_gradients.layer_id;
IF existing_weight_grad_id IS NULL THEN
– First accumulation, create accumulator
INSERT INTO gradient_accumulators (layer_id, weight_grad_matrix_id, bias_grad_vector_id,
batch_count)
VALUES (layer_id, weight_grad_matrix_id, bias_grad_vector_id, batch_size);
```

```
ELSE
– Add to existing gradients
new_weight_grad_id := sparse_matrix_add(existing_weight_grad_id, weight_grad_matrix_id);
new_bias_grad_id := sparse_vector_add(existing_bias_grad_id, bias_grad_vector_id);
UPDATE gradient_accumulators
SET weight_grad_matrix_id = new_weight_grad_id,
bias_grad_vector_id = new_bias_grad_id,
batch_count = batch_count + batch_size
WHERE gradient_accumulators.layer_id = accumulate_gradients.layer_id;
– Clean up old gradient objects (optional)
– Could be managed by cleanup_sparse_data
END IF;
END;
$$ LANGUAGE plpgsql;
– Apply gradient descent update to a layer (using accumulated gradients)
CREATE OR REPLACE FUNCTION apply_gradient_descent(
layer_id INTEGER,
learning_rate DOUBLE PRECISION
) RETURNS VOID AS $$
DECLARE
weight_m_id INTEGER;
bias_v_id INTEGER;
weight_grad_m_id INTEGER;
bias_grad_v_id INTEGER;
batch_count INTEGER;
new_weight_m_id INTEGER;
new_bias_v_id INTEGER;
scaled_weight_grad_id INTEGER;
scaled_bias_grad_id INTEGER;
BEGIN
– Get layer weights and biases
SELECT weights_matrix_id, biases_vector_id
INTO weight_m_id, bias_v_id
FROM layers WHERE layer_id = apply_gradient_descent.layer_id;
```

```
– Get accumulated gradients
SELECT weight_grad_matrix_id, bias_grad_vector_id, batch_count
INTO weight_grad_m_id, bias_grad_v_id, batch_count
FROM gradient_accumulators
WHERE gradient_accumulators.layer_id = apply_gradient_descent.layer_id;
IF weight_grad_m_id IS NULL OR bias_grad_v_id IS NULL THEN
RAISE NOTICE 'No gradients accumulated for layer %', layer_id;
RETURN;
END IF;
– Average gradients over batch
scaled_weight_grad_id := sparse_matrix_scale(weight_grad_m_id, -learning_rate / batch_count);
scaled_bias_grad_id := sparse_vector_scale(bias_grad_v_id, -learning_rate / batch_count);
– Update weights: W = W - (learning_rate / batch_size) * gradient
new_weight_m_id := sparse_matrix_add(weight_m_id, scaled_weight_grad_id);
new_bias_v_id := sparse_vector_add(bias_v_id, scaled_bias_grad_id);
– Update layer with new weights and biases
UPDATE layers
SET weights_matrix_id = new_weight_m_id,
biases_vector_id = new_bias_v_id
WHERE layer_id = apply_gradient_descent.layer_id;
– Reset gradients for this layer
PERFORM reset_gradients(layer_id);
END;
$$ LANGUAGE plpgsql;
– Reset accumulated gradients for a layer
CREATE OR REPLACE FUNCTION reset_gradients(
layer_id INTEGER
) RETURNS VOID AS $$
BEGIN
UPDATE gradient_accumulators
SET weight_grad_matrix_id = NULL,
bias_grad_vector_id = NULL,
batch_count = 0
```

```
WHERE gradient_accumulators.layer_id = reset_gradients.layer_id;
END;
$$ LANGUAGE plpgsql;
– Reset all gradients in a network
CREATE OR REPLACE FUNCTION reset_all_gradients(
network_id INTEGER
) RETURNS VOID AS $$
BEGIN
UPDATE gradient_accumulators
SET weight_grad_matrix_id = NULL,
bias_grad_vector_id = NULL,
batch_count = 0
WHERE layer_id IN (
SELECT layer_id FROM layers WHERE network_id = reset_all_gradients.network_id
);
END;
$$ LANGUAGE plpgsql;
– Training step for a single sample
– Returns (loss, accuracy) and accumulates gradients
CREATE OR REPLACE FUNCTION training_step(
sample_id INTEGER,
network_id INTEGER,
learning_rate DOUBLE PRECISION DEFAULT 0.01
) RETURNS TABLE (
loss DOUBLE PRECISION,
accuracy DOUBLE PRECISION
) AS $$
DECLARE
label_val INTEGER;
feature_v_id INTEGER;
current_input_id INTEGER;
current_output_id INTEGER;
layer_rec RECORD;
loss_val DOUBLE PRECISION;
```

```
acc_val DOUBLE PRECISION;
grad_v_id INTEGER;
weight_grad_m_id INTEGER;
bias_grad_v_id INTEGER;
input_grad_v_id INTEGER;
num_classes INTEGER;
layer_outputs INTEGER[]; – store output vector IDs per layer
BEGIN
– Get sample data
SELECT s.label, s.feature_vector_id INTO label_val, feature_v_id
FROM samples s WHERE s.sample_id = training_step.sample_id;
– Get number of classes from output layer
SELECT l.parameters-»‘output_dim’ INTO num_classes
FROM layers l
WHERE l.network_id = training_step.network_id
ORDER BY l.layer_index DESC
LIMIT 1;
– Forward pass through all layers
current_input_id := feature_v_id;
layer_outputs := ‘{}’;
FOR layer_rec IN
SELECT * FROM layers
WHERE network_id = training_step.network_id
ORDER BY layer_index
LOOP
current_output_id := forward_dense(layer_rec.layer_id, current_input_id, sample_id);
layer_outputs := array_append(layer_outputs, current_output_id);
current_input_id := current_output_id;
END LOOP;
– Compute loss and gradient at output
SELECT loss, gradient_vector_id INTO loss_val, grad_v_id
FROM softmax_cross_entropy_loss(current_output_id, label_val, num_classes::INTEGER);
– Compute accuracy
acc_val := compute_accuracy(current_output_id, label_val);
```

```
– Backward pass through layers in reverse order
FOR i IN REVERSE array_length(layer_outputs, 1)..1 LOOP
SELECT * FROM layers INTO layer_rec
WHERE network_id = training_step.network_id AND layer_index = i - 1;
SELECT weight_grad_matrix_id, bias_grad_vector_id, input_grad_vector_id
INTO weight_grad_m_id, bias_grad_v_id, input_grad_v_id
FROM backward_dense(layer_rec.layer_id, sample_id, grad_v_id);
– Accumulate gradients
PERFORM accumulate_gradients(
layer_rec.layer_id,
weight_grad_m_id,
bias_grad_v_id,
1
);
– Pass gradient to previous layer
grad_v_id := input_grad_v_id;
END LOOP;
RETURN QUERY SELECT loss_val, acc_val;
END;
$$ LANGUAGE plpgsql;
– Batch training step (process multiple samples and apply updates)
CREATE OR REPLACE FUNCTION batch_training_step(
sample_ids INTEGER[],
network_id INTEGER,
learning_rate DOUBLE PRECISION DEFAULT 0.01
) RETURNS TABLE (
avg_loss DOUBLE PRECISION,
avg_accuracy DOUBLE PRECISION
) AS $$
DECLARE
total_loss DOUBLE PRECISION := 0.0;
total_accuracy DOUBLE PRECISION := 0.0;
sample_count INTEGER := 0;
s_id INTEGER;
```

```
loss_val DOUBLE PRECISION;
acc_val DOUBLE PRECISION;
BEGIN
– Reset gradients for all layers in network
PERFORM reset_all_gradients(network_id);
– Process each sample
FOREACH s_id IN ARRAY sample_ids LOOP
SELECT loss, accuracy INTO loss_val, acc_val
FROM training_step(s_id, network_id, learning_rate);
total_loss := total_loss + loss_val;
total_accuracy := total_accuracy + acc_val;
sample_count := sample_count + 1;
END LOOP;
– Apply gradient descent to all layers
FOR layer_rec IN SELECT layer_id FROM layers WHERE network_id = batch_training_step.network_id LOOP
PERFORM apply_gradient_descent(layer_rec.layer_id, learning_rate);
END LOOP;
– Clear forward cache for these samples
DELETE FROM forward_cache WHERE sample_id = ANY(sample_ids);
IF sample_count > 0 THEN
avg_loss := total_loss / sample_count;
avg_accuracy := total_accuracy / sample_count;
ELSE
avg_loss := NULL;
avg_accuracy := NULL;
END IF;
RETURN QUERY SELECT avg_loss, avg_accuracy;
END;
$$ LANGUAGE plpgsql;– Data import functions for MNIST dataset
– Import a single MNIST sample (pixel values as array)
– Returns sample_id
CREATE OR REPLACE FUNCTION import_mnist_sample(
label INTEGER,
```

```
pixel_values INTEGER[],  – 784 pixel values (0-255)
dataset_name VARCHAR(200) DEFAULT 'mnist',
split_probability DOUBLE PRECISION DEFAULT 0.8  – probability of train vs val
) RETURNS INTEGER AS $$
DECLARE
dataset_id INTEGER;
feature_dim INTEGER := 784;
label_dim INTEGER := 10;
vector_id INTEGER;
sample_id INTEGER;
i INTEGER;
pixel_val DOUBLE PRECISION;
split_val VARCHAR(20);
BEGIN
– Get or create dataset
SELECT d.dataset_id INTO dataset_id
FROM datasets d WHERE d.name = dataset_name;
IF dataset_id IS NULL THEN
INSERT INTO datasets (name, feature_dim, label_dim, description)
VALUES (dataset_name, feature_dim, label_dim, 'MNIST handwritten digits')
RETURNING dataset_id INTO dataset_id;
END IF;
– Create sparse vector (only store pixels with value > threshold)
INSERT INTO sparse_vectors (size, metadata)
VALUES (feature_dim, jsonb_build_object('dataset', dataset_name, 'type', 'mnist_image'))
RETURNING vector_id INTO vector_id;
FOR i IN 1..array_length(pixel_values, 1) LOOP
pixel_val := pixel_values[i]::DOUBLE PRECISION / 255.0; – normalize
IF pixel_val > 0.01 THEN  – threshold to keep sparse
INSERT INTO sparse_vector_elements (vector_id, index, value)
VALUES (vector_id, i - 1, pixel_val);
END IF;
END LOOP;
– Determine split (train/val)
```

```
IF random() < split_probability THEN
split_val := 'train';
ELSE
split_val := 'val';
END IF;
– Create sample record
INSERT INTO samples (dataset_id, label, feature_vector_id, split)
VALUES (dataset_id, label, vector_id, split_val)
RETURNING sample_id INTO sample_id;
RETURN sample_id;
END;
$$ LANGUAGE plpgsql;
– Import MNIST data from a temporary table
– Assumes temporary table 'mnist_staging' exists with columns:
–   label INTEGER,
–   pixel_values INTEGER[]  – array of 784 integers
CREATE OR REPLACE PROCEDURE import_mnist_from_staging(
dataset_name VARCHAR(200) DEFAULT 'mnist',
split_probability DOUBLE PRECISION DEFAULT 0.8,
limit_rows INTEGER DEFAULT NULL
) AS $$
DECLARE
row_count INTEGER := 0;
total_rows INTEGER;
BEGIN
– Count total rows
EXECUTE 'SELECT COUNT(*) FROM mnist_staging' INTO total_rows;
RAISE NOTICE 'Importing % rows from staging table',
CASE WHEN limit_rows IS NULL THEN total_rows ELSE LEAST(limit_rows, total_rows)
END;
– Process each row
FOR row_rec IN EXECUTE
'SELECT label, pixel_values FROM mnist_staging' ||
CASE WHEN limit_rows IS NOT NULL THEN ' LIMIT ' || limit_rows ELSE ' ' END
```

```
LOOP
PERFORM import_mnist_sample(
row_rec.label,
row_rec.pixel_values,
dataset_name,
split_probability
);
row_count := row_count + 1;
IF row_count % 1000 = 0 THEN
RAISE NOTICE 'Imported % rows', row_count;
END IF;
END LOOP;
RAISE NOTICE 'Import completed: % rows imported', row_count;
END;
$$ LANGUAGE plpgsql;
– Create staging table for MNIST CSV import
CREATE OR REPLACE PROCEDURE create_mnist_staging_table() AS $$
BEGIN
– Drop if exists
DROP TABLE IF EXISTS mnist_staging;
– Create staging table with array column
CREATE TABLE mnist_staging (
label INTEGER,
pixel_values INTEGER[]  – will store all 784 pixels as array
);
RAISE NOTICE 'Created mnist_staging table';
END;
$$ LANGUAGE plpgsql;
– Load MNIST CSV file into staging table using COPY
– Note: This requires the CSV file to be accessible by PostgreSQL server
– and appropriate permissions
CREATE OR REPLACE PROCEDURE load_mnist_csv_to_staging(
csv_path TEXT,
delimiter CHAR(1) DEFAULT ','
```

) AS $$

BEGIN

– Use COPY to load data

EXECUTE format('COPY mnist_staging FROM %L WITH DELIMITER %L CSV', csv_path, delimiter);

RAISE NOTICE 'Loaded CSV file % into staging table', csv_path;

EXCEPTION

WHEN others THEN

RAISE EXCEPTION 'Failed to load CSV: %', SQLERRM;

END;

$$ LANGUAGE plpgsql;

– Simple test: create a synthetic MNIST sample

CREATE OR REPLACE FUNCTION create_test_sample(

dataset_name VARCHAR(200) DEFAULT 'mnist'

) RETURNS INTEGER AS $$

DECLARE

pixel_arr INTEGER[784];

i INTEGER;

BEGIN

– Create a simple pattern: vertical line in middle

FOR i IN 1..784 LOOP

pixel_arr[i] := 0;

END LOOP;

– Set some pixels to non-zero

pixel_arr[392] := 255; – center pixel

pixel_arr[393] := 200;

pixel_arr[394] := 150;

RETURN import_mnist_sample(5, pixel_arr, dataset_name, 1.0);

END;

$$ LANGUAGE plpgsql;– Training loop procedures for neural network training

– Create a new training run and train the network

CREATE OR REPLACE PROCEDURE train_network(

network_id INTEGER,

dataset_id INTEGER,

```
epochs INTEGER DEFAULT 10,
batch_size INTEGER DEFAULT 32,
learning_rate DOUBLE PRECISION DEFAULT 0.01,
validation_split DOUBLE PRECISION DEFAULT 0.1,
run_config JSONB DEFAULT '{}'::jsonb
) AS $$
DECLARE
run_id INTEGER;
train_sample_ids INTEGER[];
val_sample_ids INTEGER[];
total_samples INTEGER;
num_batches INTEGER;
batch_start INTEGER;
batch_end INTEGER;
batch_sample_ids INTEGER[];
epoch_idx INTEGER;
batch_idx INTEGER;
train_loss DOUBLE PRECISION;
train_accuracy DOUBLE PRECISION;
val_loss DOUBLE PRECISION;
val_accuracy DOUBLE PRECISION;
epoch_start_time TIMESTAMP;
epoch_end_time TIMESTAMP;
BEGIN
– Create training run record
INSERT INTO training_runs (network_id, dataset_id, config, status)
VALUES (
network_id,
dataset_id,
jsonb_build_object(
'epochs', epochs,
'batch_size', batch_size,
'learning_rate', learning_rate,
'validation_split', validation_split
```

) || run_config,
‘running’
)
RETURNING training_runs.run_id INTO run_id;
– Split samples into training and validation sets
– (Assuming samples already have split column populated)
SELECT ARRAY_AGG(sample_id) INTO train_sample_ids
FROM samples
WHERE dataset_id = train_network.dataset_id AND split = ‘train’;
SELECT ARRAY_AGG(sample_id) INTO val_sample_ids
FROM samples
WHERE dataset_id = train_network.dataset_id AND split = ‘val’;
IF train_sample_ids IS NULL OR array_length(train_sample_ids, 1) = 0 THEN
RAISE EXCEPTION ‘No training samples found for dataset %’, dataset_id;
END IF;
– Training loop
FOR epoch_idx IN 0..epochs-1 LOOP
epoch_start_time := CURRENT_TIMESTAMP;
– Shuffle training samples (simplified - random order)
train_sample_ids := ARRAY(
SELECT sample_id FROM samples
WHERE dataset_id = train_network.dataset_id AND split = ‘train’
ORDER BY random()
);
total_samples := array_length(train_sample_ids, 1);
num_batches := CEIL(total_samples::DOUBLE PRECISION / batch_size);
– Batch training
FOR batch_idx IN 0..num_batches-1 LOOP
batch_start := batch_idx * batch_size + 1;
batch_end := LEAST((batch_idx + 1) * batch_size, total_samples);
– Extract batch sample IDs
batch_sample_ids := train_sample_ids[batch_start:batch_end];
IF batch_sample_ids IS NOT NULL AND array_length(batch_sample_ids, 1) > 0 THEN
– Train on batch

```
SELECT avg_loss, avg_accuracy INTO train_loss, train_accuracy
FROM batch_training_step(batch_sample_ids, network_id, learning_rate);
– Optional: log batch metrics
RAISE NOTICE 'Epoch %, Batch %: loss=%, accuracy=%',
epoch_idx, batch_idx, train_loss, train_accuracy;
END IF;
END LOOP;
– Evaluate on validation set
IF val_sample_ids IS NOT NULL AND array_length(val_sample_ids, 1) > 0 THEN
SELECT avg_loss, avg_accuracy INTO val_loss, val_accuracy
FROM evaluate_network(network_id, val_sample_ids);
ELSE
val_loss := NULL;
val_accuracy := NULL;
END IF;
– Evaluate on training set (optional, can be expensive)
– For simplicity, use the last batch's metrics as proxy
– Record epoch metrics
INSERT INTO epoch_metrics (run_id, epoch, train_loss, val_loss, train_accuracy, val_accuracy)
VALUES (run_id, epoch_idx, train_loss, val_loss, train_accuracy, val_accuracy);
epoch_end_time := CURRENT_TIMESTAMP;
RAISE NOTICE 'Epoch % completed in % seconds. Train loss: %, Val loss: %, Val accuracy: %',
epoch_idx,
EXTRACT(EPOCH FROM (epoch_end_time - epoch_start_time)),
train_loss,
val_loss,
val_accuracy;
END LOOP;
– Update training run status
UPDATE training_runs
SET status = 'completed',
end_time = CURRENT_TIMESTAMP,
```

```
metrics = jsonb_build_object(
'final_train_loss', train_loss,
'final_val_loss', val_loss,
'final_val_accuracy', val_accuracy
)
WHERE training_runs.run_id = run_id;
RAISE NOTICE 'Training completed for run %', run_id;
END;
$$ LANGUAGE plpgsql;
– Evaluate network on a set of samples
CREATE OR REPLACE FUNCTION evaluate_network(
network_id INTEGER,
sample_ids INTEGER[]
) RETURNS TABLE (
avg_loss DOUBLE PRECISION,
avg_accuracy DOUBLE PRECISION
) AS $$
DECLARE
total_loss DOUBLE PRECISION := 0.0;
total_accuracy DOUBLE PRECISION := 0.0;
sample_count INTEGER := 0;
s_id INTEGER;
label_val INTEGER;
feature_v_id INTEGER;
current_input_id INTEGER;
current_output_id INTEGER;
layer_rec RECORD;
loss_val DOUBLE PRECISION;
acc_val DOUBLE PRECISION;
grad_v_id INTEGER;
num_classes INTEGER;
BEGIN
– Get number of classes from output layer
SELECT l.parameters-»'output_dim' INTO num_classes
```

```
FROM layers l
WHERE l.network_id = evaluate_network.network_id
ORDER BY l.layer_index DESC
LIMIT 1;
IF num_classes IS NULL THEN
RAISE EXCEPTION 'Could not determine number of classes from network';
END IF;
FOREACH s_id IN ARRAY sample_ids LOOP
-- Get sample data
SELECT s.label, s.feature_vector_id INTO label_val, feature_v_id
FROM samples s WHERE s.sample_id = s_id;
-- Forward pass (no caching needed for evaluation)
current_input_id := feature_v_id;
FOR layer_rec IN
SELECT * FROM layers
WHERE network_id = evaluate_network.network_id
ORDER BY layer_index
LOOP
current_output_id := forward_dense(layer_rec.layer_id, current_input_id, NULL);
current_input_id := current_output_id;
END LOOP;
-- Compute loss and accuracy
SELECT loss, gradient_vector_id INTO loss_val, grad_v_id
FROM softmax_cross_entropy_loss(current_output_id, label_val, num_classes::INTEGER);
acc_val := compute_accuracy(current_output_id, label_val);
total_loss := total_loss + loss_val;
total_accuracy := total_accuracy + acc_val;
sample_count := sample_count + 1;
END LOOP;
IF sample_count > 0 THEN
avg_loss := total_loss / sample_count;
avg_accuracy := total_accuracy / sample_count;
ELSE
avg_loss := NULL;
```

```
avg_accuracy := NULL;
END IF;
RETURN QUERY SELECT avg_loss, avg_accuracy;
END;
$$ LANGUAGE plpgsql;
– Create a simple neural network for MNIST classification
CREATE OR REPLACE PROCEDURE create_mnist_network(
network_name VARCHAR(200) DEFAULT 'mnist_classifier'
) AS $$
DECLARE
net_id INTEGER;
dense_type_id INTEGER;
BEGIN
– Get dense layer type ID
SELECT type_id INTO dense_type_id FROM layer_types WHERE name = 'dense';
IF dense_type_id IS NULL THEN
RAISE EXCEPTION 'Dense layer type not registered. Run init_layer_types() first.';
END IF;
– Create network
INSERT INTO networks (name, description, config)
VALUES (
network_name,
'Simple MLP for MNIST digit classification',
jsonb_build_object(
'input_dim', 784,
'output_dim', 10,
'hidden_layers', '[128, 64]',
'activation', 'relu',
'output_activation', 'softmax'
)
)
RETURNING network_id INTO net_id;
– Create layers
– Input layer (placeholder, not a real layer)
```

– Hidden layer 1: 784 -> 128

PERFORM create_dense_layer(net_id, 1, 128, ‘relu’, ‘he’);

– Hidden layer 2: 128 -> 64

PERFORM create_dense_layer(net_id, 2, 64, ‘relu’, ‘he’);

– Output layer: 64 -> 10 with softmax activation

– Note: softmax activation not yet implemented in sparse_vector_map

– Use linear activation and apply softmax in loss function

PERFORM create_dense_layer(net_id, 3, 10, ‘linear’, ‘xavier’);

RAISE NOTICE ‘Created MNIST network % with ID %’, network_name, net_id;

END;

$$ LANGUAGE plpgsql;

– Initialize layer types (call this once)

CREATE OR REPLACE PROCEDURE init_layer_types() AS $$

BEGIN

– Register dense layer type

INSERT INTO layer_types (name, description, forward_function, backward_function, parameter_schema)

VALUES (

‘dense’,

‘Fully connected dense layer’,

‘forward_dense’,

‘backward_dense’,

jsonb_build_object(

‘type’, ‘object’,

‘properties’, jsonb_build_object(

‘input_dim’, jsonb_build_object(‘type’, ‘integer’, ‘minimum’, 1),

‘output_dim’, jsonb_build_object(‘type’, ‘integer’, ‘minimum’, 1)

),

‘required’, jsonb_build_array(‘input_dim’, ‘output_dim’)

)

)

ON CONFLICT (name) DO UPDATE SET

forward_function = EXCLUDED.forward_function,

backward_function = EXCLUDED.backward_function,

parameter_schema = EXCLUDED.parameter_schema;

RAISE NOTICE ‘Layer types initialized’;

END;

$$ LANGUAGE plpgsql;

– Initialize layer types

CALL init_layer_types();

– Create a test network for MNIST

– CALL create_mnist_network();

1.2 🤖 Gemini

Show thinking

Gemini said

Here is the entity-relationship diagram generated from your code, designed as a poster in landscape format. Please note that while the layout and logical relationships are generally accurate—with foreign keys, unique constraints, and crow’s feet notation included—there are some minor duplication errors. Specifically, ‘sparse_vectors’ and ‘gradient_accumulators’ appear multiple times with conflicting definitions. I have included a legend to help you navigate the primary and foreign keys.

You stopped this response

1.3 👤 You

example

SH

import_mnist_gawk

SH

import_mnist.awk

Unknown

neuralsql

SH

init

SQL

test

SQL

tables

SQL

index

HTML

server

TS

You said

Explain how a neural network including training can work using mostly SQL and simple CLI bash scripts. Write this as a detailed report based on the given context and files.

1.4 🤖 Gemini

Show thinking

Gemini said

NeuralSQL: Relational Database as a Differentiable Computational Engine

The implementation of a differentiable computational graph entirely within a relational database management system (RDBMS) requires mapping standard tensor operations to relational algebra. By leveraging PostgreSQL's PL/pgSQL procedural language and standard SQL JOIN constraints, we can execute forward and backward passes, manage state, and compute gradients without relying on external array-processing libraries or languages.

Below is the technical breakdown of the NeuralSQL architecture, demonstrating how a Multi-Layer Perceptron (MLP) is constructed and optimized.

1. Relational Representation of Sparse Tensors

Traditional deep learning frameworks allocate contiguous blocks of memory for tensors. NeuralSQL utilizes a sparse representation distributed across normalized tables, which inherently optimizes for zero-valued elements typical in functions like ReLU or zero-padded data.

Tensor metadata and discrete non-zero elements are decoupled:

Vectors: Defined in sparse_vectors (tracking vector dimensionality). Non-zero elements are stored in sparse_vector_elements with explicit integer bounds CHECK (index >= 0) and CHECK (index < size).

Matrices: Defined in sparse_matrices (tracking row/column dimensions). Elements reside in sparse_matrix_elements mapped by (row, col) tuples.

This schema guarantees that memory scales with the informational density of the network rather than its absolute parameter count.

2. Linear Algebra via Relational Algebra

The core operation of a dense neural network layer is the affine transformation:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

In NeuralSQL, matrix-vector multiplication is executed as an inner join combined with an aggregate grouping. The computational graph maps the column index of the weight matrix **W** to the element index of the input vector **x**:

```sql
INSERT INTO sparse_vector_elements (vector_id, index, value)
SELECT result_id, m.row AS index, SUM(m.value * v.value) AS value
FROM sparse_matrix_elements m
JOIN sparse_vector_elements v ON m.col = v.index
WHERE m.matrix_id = $1 AND v.vector_id = $2
GROUP BY m.row
HAVING ABS(SUM(m.value * v.value)) > 1e-15;
```

Proof of Operation: The JOIN isolates pairs where W_{ij} and x_j are both non-zero. The SUM aggregate computes the dot product for each row i, yielding the pre-activation vector $\mathbf{z}$.

3. State Caching and Forward Propagation

To compute gradients during backpropagation, intermediate tensor states must be preserved.

The forward_dense function coordinates the forward pass for a specific layer. It executes the affine transformation and maps a non-linear activation function (e.g., ReLU, Sigmoid) over the resulting vector. Crucially, it caches the inputs and pre-activations into the forward_cache table, using (layer_id, sample_id) as a unique composite key. This acts as the differentiable state tape.

4. Backpropagation and the Chain Rule

The backward_dense function computes parameter gradients using the cached states. Given an upstream gradient from the subsequent layer or loss function ($\nabla_{\mathbf{a}} L$), the system applies the chain rule.

Activation Gradient: Computes the derivative of the activation function with respect to the pre-activation $\mathbf{z}$. For ReLU, this operates sparsely by projecting 1.0 where $\mathbf{z} > 0$.

Local Error (δ): Computes the Hadamard product ($\odot$) of the upstream gradient and the activation gradient via an INNER JOIN on matching indices.

$$\delta = \nabla_{\mathbf{a}} L \odot \sigma'(\mathbf{z})$$

Weight Gradient: Computes the outer product $\delta \mathbf{x}^T$ via a CROSS JOIN between the δ vector elements and the cached input vector $\mathbf{x}$ elements.

Input Gradient: Propagates the error backwards ($\mathbf{W}^T \delta$) using a transposed matrix-vector multiplication (grouping by m.col instead of m.row).

5. Gradient Accumulation and Optimization

Mini-batch gradient descent requires accumulating gradients across multiple independent samples before updating the weights. NeuralSQL manages this statefully via the gradient_accumulators table.

The batch_training_step procedure iterates over a designated array of sample IDs, accumulating the computed gradients. Once the batch completes, apply_gradient_descent executes the parameter update:

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \frac{\alpha}{N} \sum_{k=1}^{N} \nabla_{\mathbf{W}_k} L$$

This is implemented by scaling the accumulated gradient matrix by $-\frac{\alpha}{N}$ and executing a FULL OUTER JOIN to add it to the existing weight matrix.

6. Pipeline Orchestration via CLI

Bash scripts provide the execution environment, replacing high-level frameworks like Python.

Ingestion: The script import_mnist_gawk.sh utilizes gawk to stream CSV text processing, generating optimized SQL INSERT statements into a staging table, circumventing memory bottlenecks during data load.

Execution: neuralsql.sh acts as the command dispatcher, wrapping database calls (psql) into standard arguments (e.g., init, create-network, train).

Telemetry: Training metrics (loss, accuracy) are stored immutably in epoch_metrics. A lightweight Bun-based TypeScript server (server.ts) proxies a PostgREST API to expose these metrics to an HTML frontend, leveraging Plotly for real-time visualization.

Would you like me to draft a Bun script that automates the generation of dynamic SQL queries for hyperparameter grid search across this database architecture?

You stopped this response
